High Level API: TextDetectionModel and TextRecognitionModel

Original author	Wenqing Zhang
Compatibility	OpenCV >= 4.5

Introduction

In this tutorial, we will introduce the APIs for TextRecognitionModel and TextDetectionModel in detail.

TextRecognitionModel

In the current version, cv::dnn::TextRecognitionModel only supports CNN+RNN+CTC based algorithms, and the greedy decoding method for CTC is provided. For more information, please refer to the original paper

Before recognition, you should `setVocabulary` and `setDecodeType`.

- "CTC-greedy", the output of the text recognition model should be a probability matrix. The shape should be `(T, B, Dim)`, where
    - T is the sequence length
    - B is the batch size (only support `B=1` in inference)
    - and `Dim` is the length of vocabulary +1('Blank' of CTC is at the index=0 of Dim).
- "CTC-prefix-beam-search", the output of the text recognition model should be a probability matrix same with "CTC-greedy".
    - The algorithm is proposed at Hannun's paper.
    - `setDecodeOptsCTCPrefixBeamSearch` could be used to control the beam size in search step.
    - To further optimize for big vocabulary, a new option `vocPruneSize` is introduced to avoid iterate the whole vocbulary but only the number of `vocPruneSize` tokens with top probability.

cv::dnn::TextRecognitionModel::recognize() is the main function for text recognition.

- The input image should be a cropped text image or an image with `roiRects`
- Other decoding methods may supported in the future

TextDetectionModel

cv::dnn::TextDetectionModel API provides these methods for text detection:

- cv::dnn::TextDetectionModel::detect() returns the results in std::vector<std::vector<Point>> (4-points quadrangles)
- cv::dnn::TextDetectionModel::detectTextRectangles() returns the results in std::vector<cv::RotatedRect> (RBOX-like)

In the current version, cv::dnn::TextDetectionModel supports these algorithms:

- use cv::dnn::TextDetectionModel_DB with "DB" models
- and use cv::dnn::TextDetectionModel_EAST with "EAST" models

The following provided pretrained models are variants of DB (w/o deformable convolution), and the performance can be referred to the Table.1 in the paper. For more information, please refer to the official code

---

You can train your own model with more data, and convert it into ONNX format. We encourage you to add new algorithms to these APIs.

Pretrained Models

TextRecognitionModel

crnn.onnx:
url: https://drive.google.com/uc?export=dowload&id=1ooaLR-rkTl8jdpGy1DoQs0-X0lQsB6Fj
sha: 270d92c9ccb670ada2459a25977e8deeaf8380d3,
alphabet_36.txt: https://drive.google.com/uc?
export=dowload&id=1oPOYx5rQRp8L6XQciUwmwhMCfX0KyO4b
parameter setting: -rgb=0;
description: The classification number of this model is 36 (0~9 + a~z).
The training dataset is MJSynth.
crnn_cs.onnx:
url: https://drive.google.com/uc?export=dowload&id=12diBsVJrS9ZEl6BNUiRp9s0xPALBS7kt
sha: a641e9c57a5147546f7a2dbea4fd322b47197cd5
alphabet_94.txt: https://drive.google.com/uc?
export=dowload&id=1oKXxXKusquimp7XY1mFvj9nwLzldVgBR
parameter setting: -rgb=1;
description: The classification number of this model is 94 (0~9 + a~z + A~Z + punctuations).
The training datasets are MJsynth and SynthText.
crnn_cs_CN.onnx:
url: https://drive.google.com/uc?export=dowload&id=1is4eYEUKH7HR7Gl37Sw4WPXx6Ir8oQEG
sha: 3940942b85761c7f240494cf662dcbf05dc00d14
alphabet_3944.txt: https://drive.google.com/uc?
export=dowload&id=18IZUUdNzJ44heWTndDO6NNfIpJMmN-ul
parameter setting: -rgb=1;

description: The classification number of this model is 3944 (0~9 + a~z + A~Z + Chinese characters + special characters).
The training dataset is ReCTS (https://rrc.cvc.uab.es/?ch=12).

More models can be found in here, which are taken from clovaai. You can train more models by CRNN, and convert models by `torch.onnx.export`.

TextDetectionModel

- DB_IC15_resnet50.onnx:
url: https://drive.google.com/uc?export=dowload&id=17_ABp79PlFt9yPCxSaarVc_DKTmrSGGf
sha: bef233c28947ef6ec8c663d20a2b326302421fa3
recommended parameter setting: -inputHeight=736, -inputWidth=1280;
description: This model is trained on ICDAR2015, so it can only detect English text instances.
- DB_IC15_resnet18.onnx:
url: https://drive.google.com/uc?export=dowload&id=1vY_KsDZZZb_svd5RT6pjyI8BS1nPbBSX
sha: 19543ce09b2efd35f49705c235cc46d0e22df30b
recommended parameter setting: -inputHeight=736, -inputWidth=1280;
description: This model is trained on ICDAR2015, so it can only detect English text instances.
- DB_TD500_resnet50.onnx:
url: https://drive.google.com/uc?export=dowload&id=19YWhArrNccaoSza0CfkXlA8im4-lAGsR
sha: 1b4dd21a6baa5e3523156776970895bd3db6960a
recommended parameter setting: -inputHeight=736, -inputWidth=736;
description: This model is trained on MSRA-TD500, so it can detect both English and Chinese text instances.
- DB_TD500_resnet18.onnx:
url: https://drive.google.com/uc?export=dowload&id=1sZszH3pEt8hliyBlTmB-iulxHP1dCQWV
sha: 8a3700bdc13e00336a815fc7afff5dcc1ce08546
recommended parameter setting: -inputHeight=736, -inputWidth=736;
description: This model is trained on MSRA-TD500, so it can detect both English and Chinese text instances.

We will release more models of DB here in the future.

- EAST:
Download link: https://www.dropbox.com/s/r2ingd0l3zt8hxs/frozen_east_text_detection.tar.gz?dl=1
This model is based on https://github.com/argman/EAST

Images for Testing

Text Recognition:
url: https://drive.google.com/uc?export=dowload&id=1nMcEy68zDNpIlqAn6xCk_kYcUTIeSOtN
sha: 89205612ce8dd2251effa16609342b69bff67ca3
Text Detection:

url: https://drive.google.com/uc?export=dowload&id=149tAhIcvfCYeyufRoZ9tmc2mZDKE_XrF
sha: ced3c03fb7f8d9608169a913acf7e7b93e07109b

Example for Text Recognition

Step1. Loading images and models with a vocabulary

```
// Load a cropped text line image
// you can find cropped images for testing in "Images for Testing"
int rgb = IMREAD_COLOR; // This should be changed according to the model input requirement.
Mat image = imread("path/to/text_rec_test.png", rgb);
// Load models weights
TextRecognitionModel model("path/to/crnn_cs.onnx");
// The decoding method
// more methods will be supported in future
model.setDecodeType("CTC-greedy");
// Load vocabulary
// vocabulary should be changed according to the text recognition model
std::ifstream vocFile;
vocFile.open("path/to/alphabet_94.txt");
CV_Assert(vocFile.is_open());
String vocLine;
std::vector<String> vocabulary;
while (std::getline(vocFile, vocLine)) {
vocabulary.push_back(vocLine);
}
model.setVocabulary(vocabulary);
```

Step2. Setting Parameters

```
// Normalization parameters
double scale = 1.0 / 127.5;
Scalar mean = Scalar(127.5, 127.5, 127.5);
// The input shape
Size inputSize = Size(100, 32);
model.setInputParams(scale, inputSize, mean);
```

Step3. Inference

```
std::string recognitionResult = recognizer.recognize(image);
std::cout << "" << recognitionResult << "" << std::endl;
```

Input image:

Picture example

Output:

'welcome'

Example for Text Detection

Step1. Loading images and models

```
// Load an image
// you can find some images for testing in "Images for Testing"
Mat frame = imread("/path/to/text_det_test.png");
```

Step2.a Setting Parameters (DB)

```
// Load model weights
TextDetectionModel_DB model("/path/to/DB_TD500_resnet50.onnx");
// Post-processing parameters
float binThresh = 0.3;
float polyThresh = 0.5;
uint maxCandidates = 200;
double unclipRatio = 2.0;
model.setBinaryThreshold(binThresh)
.setPolygonThreshold(polyThresh)
.setMaxCandidates(maxCandidates)
.setUnclipRatio(unclipRatio)
;
// Normalization parameters
double scale = 1.0 / 255.0;
Scalar mean = Scalar(122.67891434, 116.66876762, 104.00698793);
// The input shape
Size inputSize = Size(736, 736);
model.setInputParams(scale, inputSize, mean);
Step2.b Setting Parameters (EAST)
TextDetectionModel_EAST model("EAST.pb");
float confThreshold = 0.5;
float nmsThreshold = 0.4;
model.setConfidenceThreshold(confThresh)
.setNMSThreshold(nmsThresh)
;
double detScale = 1.0;
Size detInputSize = Size(320, 320);
Scalar detMean = Scalar(123.68, 116.78, 103.94);
bool swapRB = true;
model.setInputParams(detScale, detInputSize, detMean, swapRB);
```

Step3. Inference

```
std::vector<std::vector<Point>> detResults;
model.detect(detResults);
// Visualization
polylines(frame, results, true, Scalar(0, 255, 0), 2);
imshow("Text Detection", image);
waitKey();
```

Output:

Example for Text Spotting

After following the steps above, it is easy to get the detection results of an input image. Then, you can do transformation and crop text images for recognition. For more information, please refer to **Detailed Sample**

```
// Transform and Crop
Mat cropped;
fourPointsTransform(recInput, vertices, cropped);
String recResult = recognizer.recognize(cropped);
```

Output Examples:

Source Code

The [source code](https://github.com/opencv/opencv/blob/4.x/modules/dnn/src/model.cpp) of these APIs can be found in the DNN module.

`https://github.com/opencv/opencv/blob/4.x/modules/dnn/src/model.cpp`

Detailed Sample

For more information, please refer to:

- samples/dnn/scene_text_recognition.cpp
- samples/dnn/scene_text_detection.cpp
- samples/dnn/text_detection.cpp
- samples/dnn/scene_text_spotting.cpp

Test with an image

Examples:

```
example_dnn_scene_text_recognition -mp=path/to/crnn_cs.onnx -i=path/to/an/image
-rgb=1 -vp=/path/to/alphabet_94.txt
example_dnn_scene_text_detection -mp=path/to/DB_TD500_resnet50.onnx
-i=path/to/an/image -ih=736 -iw=736
example_dnn_scene_text_spotting -dmp=path/to/DB_IC15_resnet50.onnx
-rmp=path/to/crnn_cs.onnx -i=path/to/an/image -iw=1280 -ih=736 -rgb=1
-vp=/path/to/alphabet_94.txt
example_dnn_text_detection -dmp=path/to/EAST.pb -rmp=path/to/crnn_cs.onnx
-i=path/to/an/image -rgb=1 -vp=path/to/alphabet_94.txt
```

Test on public datasets

Text Recognition:

The download link for testing images can be found in the **Images for Testing**

Examples:

```
example_dnn_scene_text_recognition -mp=path/to/crnn.onnx -e=true
-edp=path/to/evaluation_data_rec -vp=/path/to/alphabet_36.txt -rgb=0
example_dnn_scene_text_recognition -mp=path/to/crnn_cs.onnx -e=true
-edp=path/to/evaluation_data_rec -vp=/path/to/alphabet_94.txt -rgb=1
```

Text Detection:

The download links for testing images can be found in the **Images for Testing**

Examples:

```
example_dnn_scene_text_detection -mp=path/to/DB_TD500_resnet50.onnx -e=true
-edp=path/to/evaluation_data_det/TD500 -ih=736 -iw=736
example_dnn_scene_text_detection -mp=path/to/DB_IC15_resnet50.onnx -e=true
-edp=path/to/evaluation_data_det/IC15 -ih=736 -iw=1280
```